

Mechanistic Interpretability

CS60216: Safety Fundamentals for Generative AI

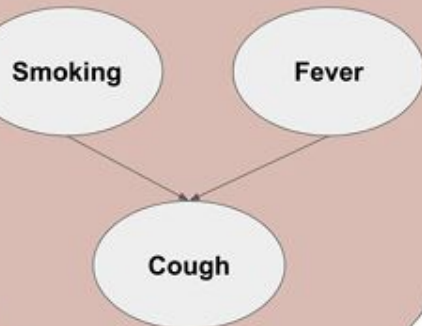
Interpretability (in AI)

Definition: *The study of understanding the decisions that AI systems make and putting them into easily human-understandable terms.*

Why: to use that understanding to iteratively better design systems that are more *performant* and *human-understandable*.

Historically models were small

BAYES NETS



LINEAR REGRESSION

$$y = mx + b$$

MULTIVARIATE LINEAR REGRESSION

$$y = Wx + b$$

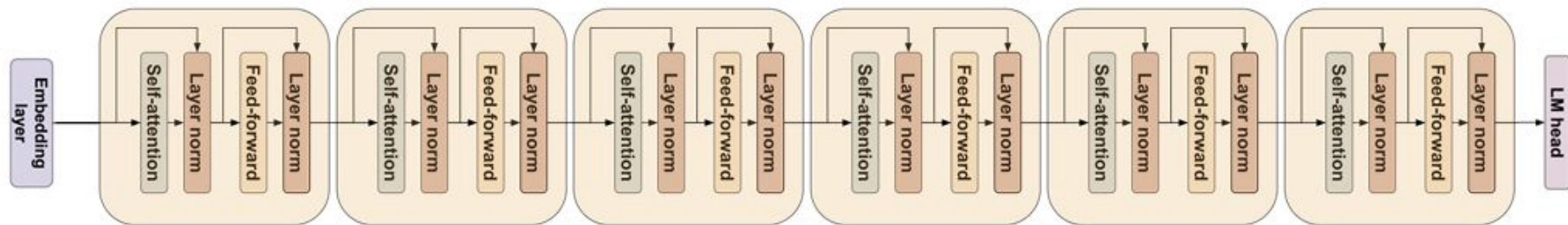
LOGISTIC REGRESSION

$$\text{sigmoid}(Wx + b)$$

2 LAYER MLP w/ ReLU

$$W(\text{ReLU}(Wx + b)) + c$$

How do we make sense of this huge model?



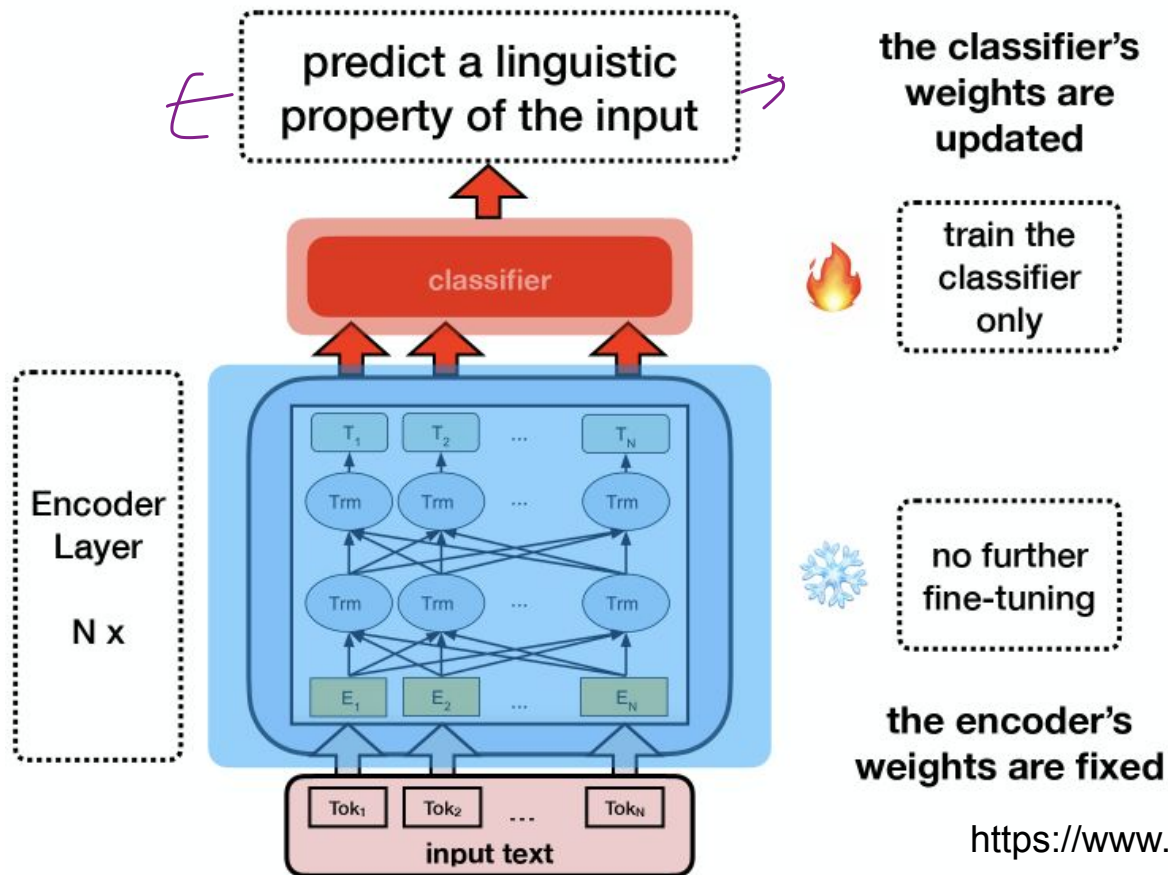
First attempt: Probing

2018-2019

Definition: *A classifier that is specifically trained to predict some property from a pretrained model's representations.*

How? Given a pretrained model, we use the representations it produces to train a classifier (without further fine-tuning the model) to predict a linguistic property of the input text

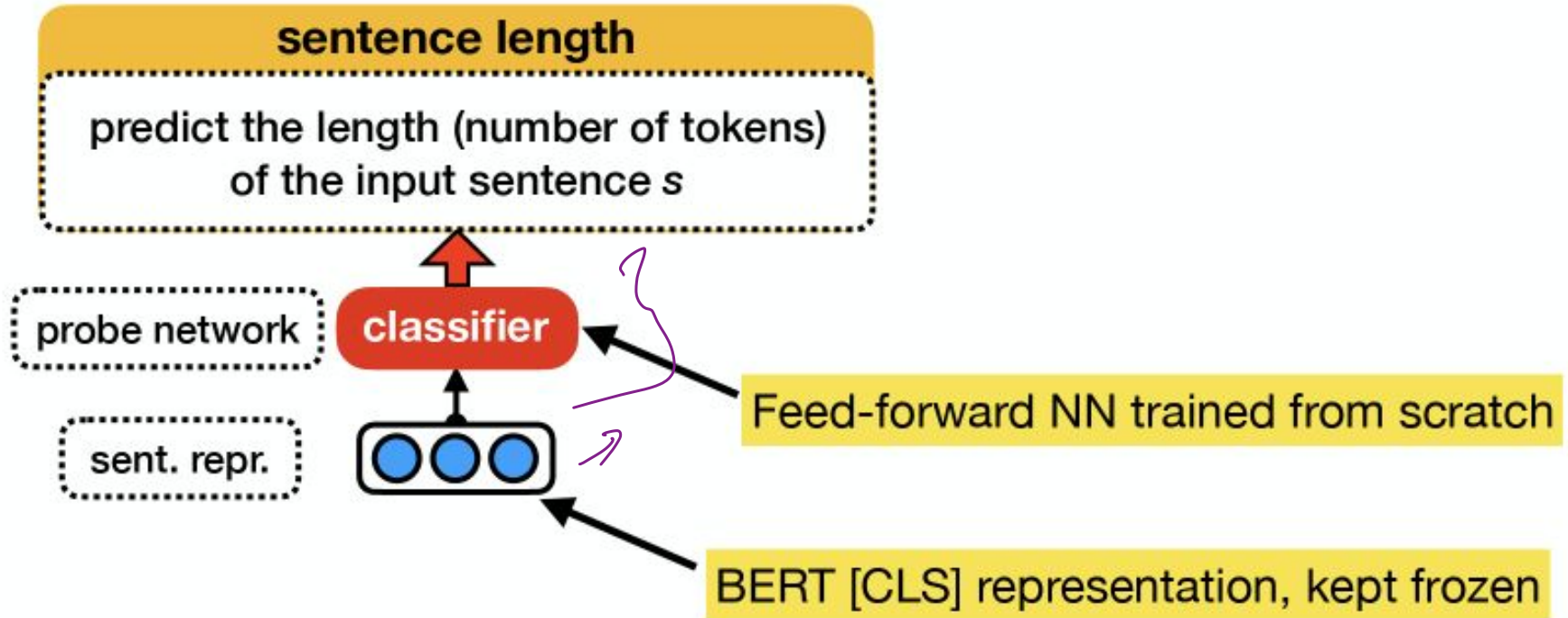
Probe Approach



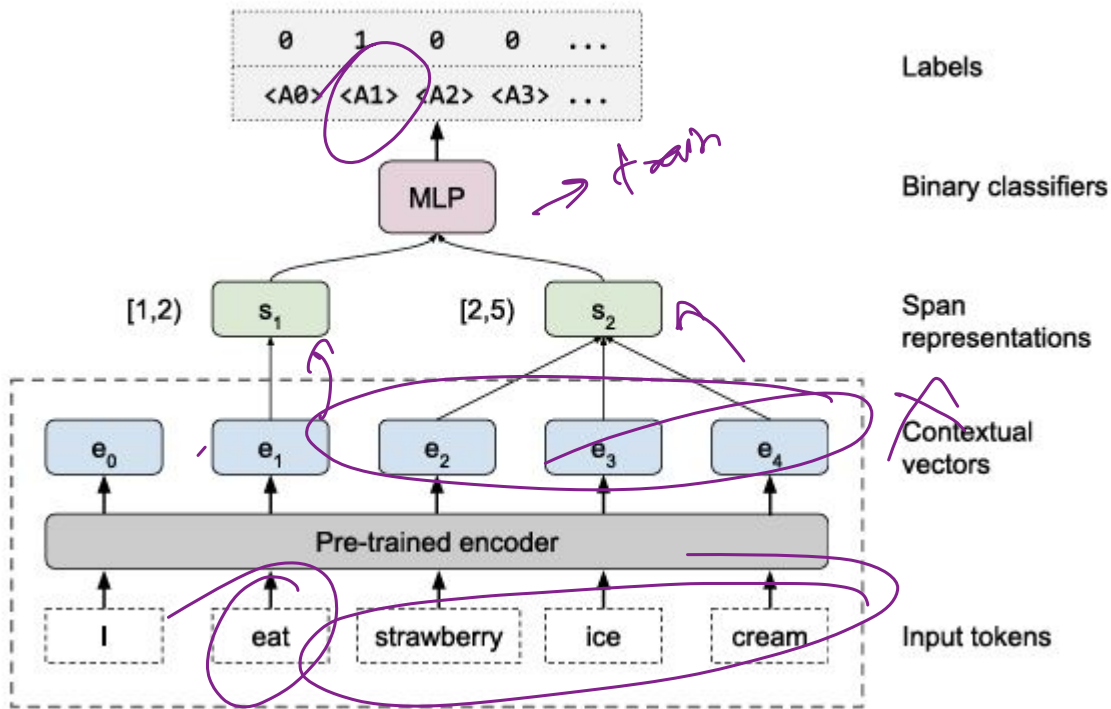
Motivation of probe tasks

- if we can train a classifier to predict a property of the input text based on its representation, it means the property is encoded somewhere in the representation
- if we cannot train a classifier to predict a property of the input text based on its representation, it means the property is not encoded in the representation or not encoded in a useful way.

Probing



Edge Probing



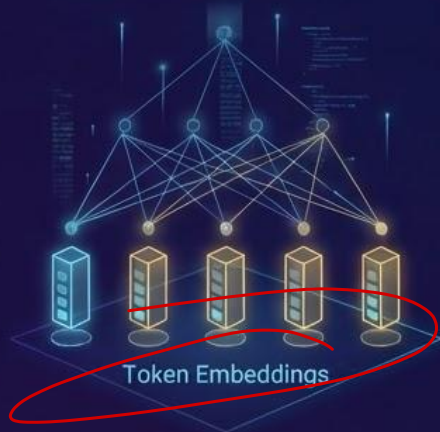
All parameters inside the dashed line are fixed, while we train the span pooling and MLP classifiers to extract information from the contextual vectors. The example shown is for semantic role labeling, where $s(1) = [1, 2)$ corresponds to the predicate ("eat"), while $s(2) = [2, 5)$ is the argument ("strawberry ice cream"), and we predict label $A1$ as positive and others as negative. For entity and constituent labeling, only a single span is used.

Edge Probing

POS	The important thing about Disney is that it is a global [brand] ₁ . → NN (Noun)
Constit.	The important thing about Disney is that it [is a global brand] ₁ . → VP (Verb Phrase)
Depend.	[Atmosphere] ₁ is always [fun] ₂ → nsubj (nominal subject)
Entities	The important thing about [Disney] ₁ is that it is a global brand. → Organization
SRL	[The important thing about Disney] ₂ [is] ₁ that it is a global brand. → Arg1 (Agent)
SPR	[It] ₁ [endorsed] ₂ the White House strategy. . . → {awareness, existed_after, . . . }
Coref. ^O	The important thing about [Disney] ₁ is that [it] ₂ is a global brand. → True
Coref. ^W	[Characters] ₂ entertain audiences because [they] ₁ want people to be happy. → True Characters entertain [audiences] ₂ because [they] ₁ want people to be happy. → False
Rel.	The [burst] ₁ has been caused by water hammer [pressure] ₂ . → Cause-Effect(e_2, e_1)

How to combine the representations across layers?

Lexical Baselines



For GPT and for BERT, we use the learned subword embeddings from the full model.

Contextual Vectors

cat (Concatenation)

Subword
Embeddings

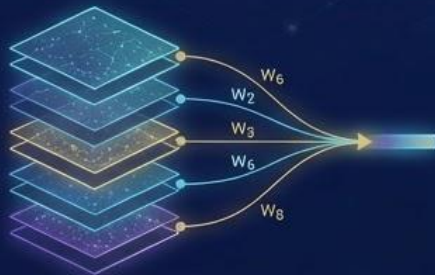


Top Layer
Activations



Concatenate the subword embeddings with the activations of the top layer.

mix (Linear Combination / ELMo-style)



$$\text{ELMo}_\theta^{\text{task}} = E(R_\theta; \Theta^{\text{task}}) = \gamma^{\text{task}} \sum_{j=0}^L s_j^{\text{task}} \mathbf{h}_{\theta,j}^{\text{LM}}.$$

Take a linear combination of layer activations (including embeddings) using learned task-specific scalars, similar to ELMo.

Performance Evaluation

F1 Scores & Differences

Task	BERT-base				BERT-large				
	Lex.	cat	mix	Abs. Δ ELMo	Lex.	cat	mix	(base) ELMo	Abs. Δ ELMo
Part-of-Speech	88.4	97.0	96.7	0.0	88.1	96.5	96.9	0.2	0.2
Constituents	68.4	83.7	86.7	2.1	69.0	80.1	87.0	0.4	2.5
Dependencies	80.1	93.0	95.1	1.1	80.2	91.5	95.4	0.3	1.4
Entities	90.9	96.1	96.2	0.6	91.8	96.2	96.5	0.6	0.9
SRL (all)	75.4	89.4	91.3	1.2	76.5	88.2	92.3	1.0	2.2
Core roles	74.9	91.4	93.6	1.0	76.3	89.9	94.6	1.0	2.0
Non-core roles	76.4	84.7	85.9	1.8	76.9	84.1	86.9	1.0	2.8
OntoNotes coref.	74.9	88.7	90.2	6.3	75.7	89.6	91.4	1.2	7.4
SPR1	79.2	84.7	86.1	1.3	79.6	85.1	85.8	-0.3	1.0
SPR2	81.7	83.0	83.8	0.7	81.6	83.2	84.1	0.3	1.0
Winograd coref.	54.3	53.6	54.9	1.4	53.0	53.8	61.4	6.5	7.8
Rel. (SemEval)	57.4	78.3	82.0	4.2	56.2	77.6	82.4	0.5	4.6
Macro Average	75.1	84.8	86.3	1.9	75.2	84.2	87.3	1.0	2.9

Key Finding

Concatenation (cat)



Scalar Mixing (mix)

Improves
performance
significantly

**ELMo-style scalar mixing (mix)
instead of concatenation improves
performance significantly**

BERT rediscovers the NLP pipeline (Tenney et al. 2019)

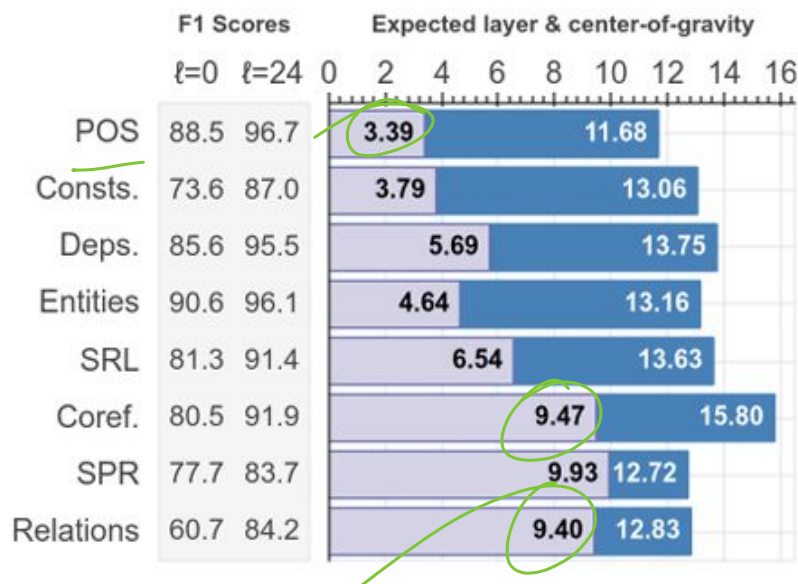


Figure 1: Summary statistics on BERT-large. Columns on left show F1 dev-set scores for the baseline ($P_{\tau}^{(0)}$) and full-model ($P_{\tau}^{(L)}$) probes. Dark (blue) are the mixing weight center of gravity (Eq. 2); light (purple) are the expected layer from the cumulative scores (Eq. 4).

POS - part of speech tagging (e.g. this word is a noun)

consts - constituent labeling (e.g. is this span a noun phrase)

deps - dependency labeling (e.g. is span_one the subject and span_two the object)

entities - named entity labeling (e.g. this word is a person)

SRL - semantic role labeling (what roles are the spans playing with each other: "Mary (pusher) pushed John (pushee)")

coref - coreference (do span_one and span_two refer to the same entity or event)

SPR - semantic proto-role (identifying attributes like awareness so is Mary aware that they are doing the pushing)

relations - relation classification (predicting the real-world relation between two spans given a set of these)

<https://aclanthology.org/P19-1452.pdf>

BERT rediscovers the NLP pipeline (Tenney et al. 2019)

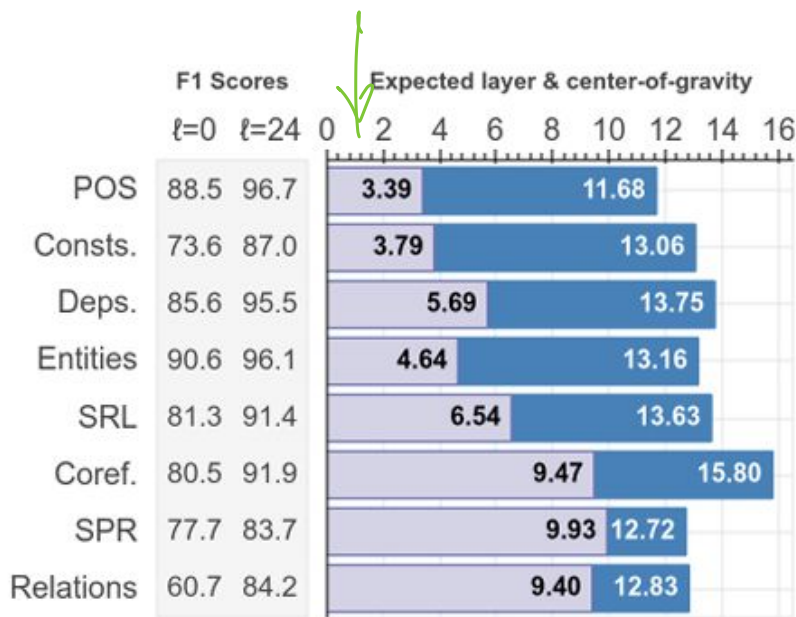


Figure 1: Summary statistics on BERT-large. Columns on left show F1 dev-set scores for the baseline ($P_{\tau}^{(0)}$) and full-model ($P_{\tau}^{(L)}$) probes. Dark (blue) are the mixing weight center of gravity (Eq. 2); light (purple) are the expected layer from the cumulative scores (Eq. 4).

Representation for the probe:

$$\mathbf{h}_{i,\tau} = \gamma_{\tau} \sum_{\ell=0}^L s_{\tau}^{(\ell)} \mathbf{h}_i^{(\ell)}$$

We learn these weights jointly with the probing classifier in order to allow it to extract information from the many layers of an encoder without adding a large number of parameters.

After the probing model is trained, we extract the learned coefficients in order to estimate the contribution of different layers to that particular task.

BERT rediscovers the NLP pipeline (Tenney et al. 2019)

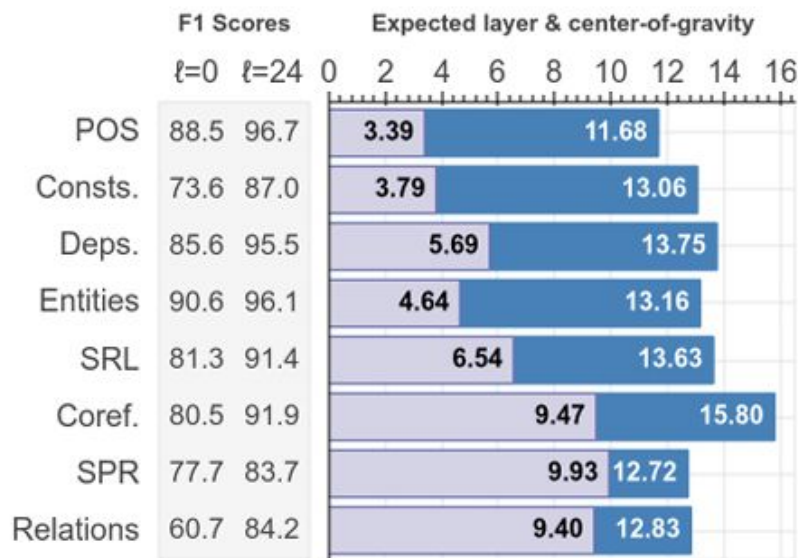


Figure 1: Summary statistics on BERT-large. Columns on left show F1 dev-set scores for the baseline ($P_{\tau}^{(0)}$) and full-model ($P_{\tau}^{(L)}$) probes. Dark (blue) are the mixing weight center of gravity (Eq. 2); light (purple) are the expected layer from the cumulative scores (Eq. 4).

$\lceil \quad \ell=1 \quad - \quad - \quad 24 \quad$
 $\Delta_{\tau}^{(\ell)} = \text{Score}(P_{\tau}^{(\ell)}) - \text{Score}(P_{\tau}^{(\ell-1)})$

measures how much better we do on the probing task if we observe one additional encoder layer ℓ :

$$\bar{E}_{\Delta}[\ell] = \frac{\sum_{\ell=1}^L \ell \cdot \Delta_{\tau}^{(\ell)}}{\sum_{\ell=1}^L \Delta_{\tau}^{(\ell)}}$$

This can be thought of as, approximately, the expected layer at which the probing model correctly labels an example, assuming that example is resolved at *some* layer $\ell \geq 1$ of the encoder.

Why does in-context learning work?

In-context learning means language models learning to:



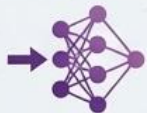
do **new tasks**



better **predict tokens**, or



generally **reduce their loss**



during the **forward-pass** at inference-time,



without any gradient-based updates to the model's parameters.

A Mathematical Framework for Transformer Circuits

AUTHORS

Nelson Elhage^{*,†}, Neel Nanda^{*}, Catherine Olsson^{*}, Tom Henighan[†], Nicholas Joseph[†], Ben Mann[†], Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, Chris Olah^{*}

^{*} Core Research Contributor; [†] Core Infrastructure Contributor; ^{*} Correspondence to colah@anthropic.com;
Author contributions statement below.

AFFILIATION

Anthropic

PUBLISHED

Dec 22, 2021



<https://transformer-circuits.pub/2021/framework/index.html>

Induction Head Hypothesis

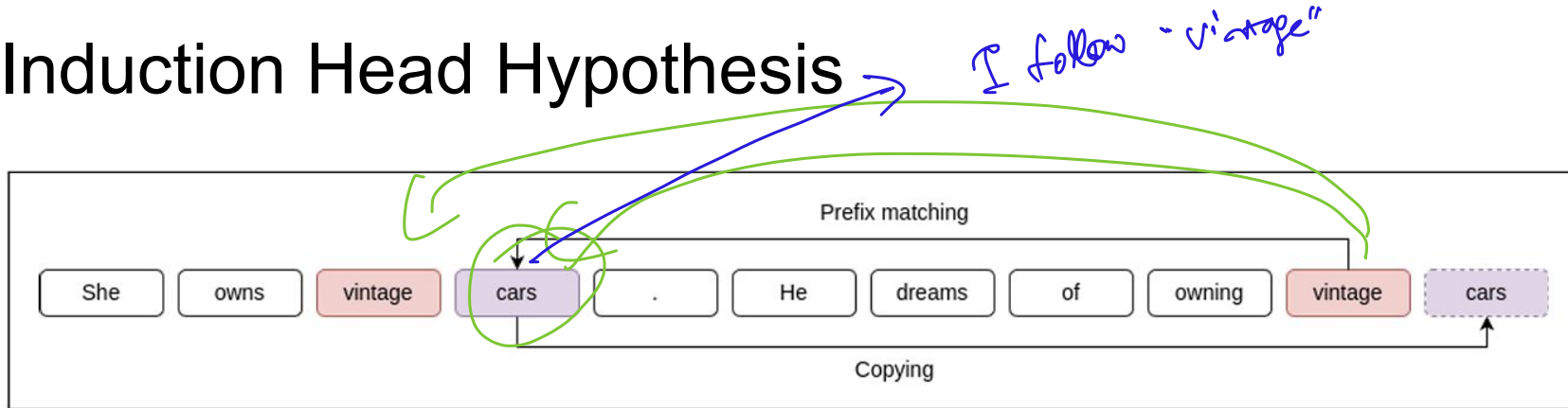


Figure 12.3 An induction head looking at **vintage** uses the *prefix matching* mechanism to find a prior instance of **vintage**, and the *copying* mechanism to predict that **cars** will occur again. Figure from [Crosbie and Shutova \(2022\)](#).

The function of the induction head is to predict repeated sequences.

For example if it sees the pattern $A B \dots A$ in an input sequence, it predicts that B will follow, instantiating the pattern completion rule $A B \dots A B$.

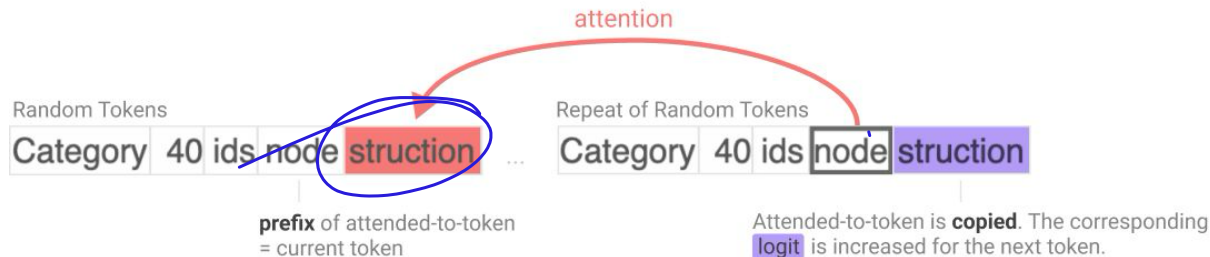
Induction head: the formal definition

Formally, **we define an induction head** as one which exhibits the following two properties on a repeated random sequence of tokens:

- **Prefix matching:** The head attends back to previous tokens that were followed by the current and/or recent tokens. That is, it attends to the token which induction would suggest comes next.

- **Copying:** The head's output increases the logit corresponding to the attended-to token.

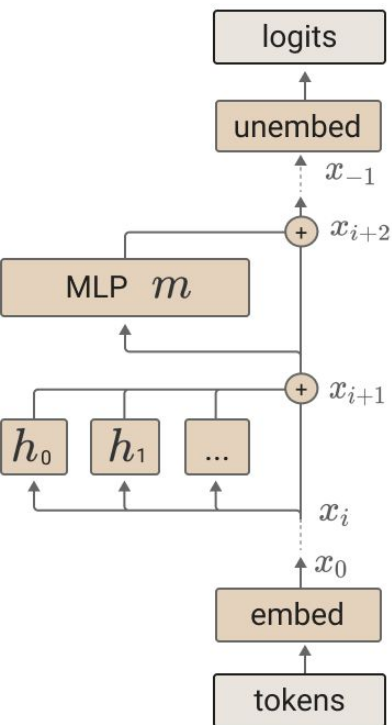
In other words, induction heads are any heads that empirically increase the likelihood of [B] given [A][B]...[A] when shown a repeated sequence of completely random tokens. An illustration of induction heads' behavior is shown here:



Induction Head Hypothesis

- Induction heads are the name for a circuit, which is a kind of abstract component of a network. ✓
- The induction head circuit is part of the attention computation in transformers, discovered by looking at mini language models with only 1-2 attention heads.

Transformer Architecture



The final logits are produced by applying the unembedding.

$$T(t) = W_U x_{-1}$$

An MLP layer, m , is run and added to the residual stream.

$$x_{i+2} = x_{i+1} + m(x_{i+1})$$

Each attention head, h , is run and added to the residual stream.

$$x_{i+1} = x_i + \sum_{h \in H_i} h(x_i)$$

One
residual
block

Token embedding.

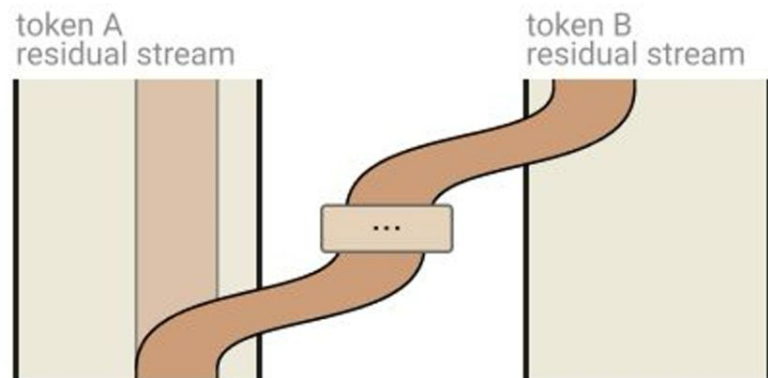
$$x_0 = W_E t$$

Residual stream is simply the sum of the output of all the previous layers and the original embedding.

The residual stream

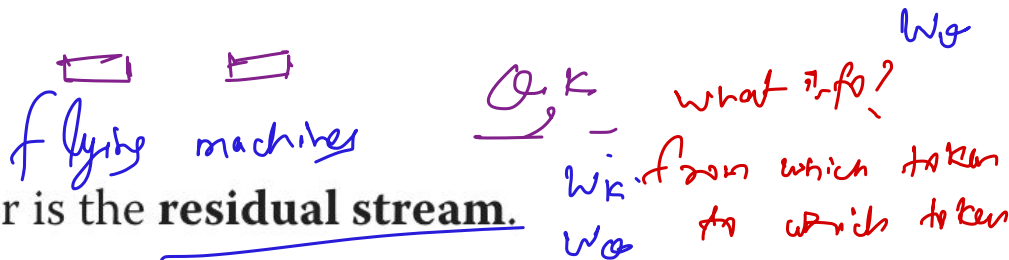
Attention heads move information across residual streams.

MLPs move information across dimensions within a residual stream.



Attention heads copy information from the residual stream of one token to the residual stream of another. They typically write to a different subspace than they read from.

Important notions


f lyng machines Q, K W_g
 W_k what info?
 W_g from which token
to which token

- The central object in the transformer is the **residual stream**.
- Different heads in each layer can be thought of as **operating independently** of each other, reading and writing into the residual stream.
- Heads can compose to form **circuits**. For instance, **K -composition** is when the output of one head is used to generate the key vector in the attention calculations of a subsequent head.
- We can describe the weight matrices W_Q , W_K and W_V as reading from (or **projecting from**) the residual stream, and W_O as writing to (or **embedding into**) the residual stream.


 QK OV

Important notions

- We can think of the combined operations W_Q and W_K in terms of a single, low-rank matrix $W_{QK} := W_Q W_K^T$, called the **QK circuit**.¹
 - This matrix defines a bilinear form on the vectors in the residual stream:
 $v_i^T W_{QK} v_j$ is the attention paid by the i th token to the j th token.
 - Conceptually, this matrix tells us **which tokens information is moved to & from** in the residual stream.
- We can think of the combined operations W_V and W_O in terms of a single matrix $W_{OV} := W_V W_O$, called the **OV circuit**.
 - This matrix defines a map from residual stream vectors to residual stream vectors: if v_j is the residual stream vector at the source token, then $v_j^T W_{OV}$ is the vector that gets moved from token j to the destination token (if j is attended to).
 - Conceptually, this matrix tells us **what information is moved from a token**, if that token is attended to.

Subspaces in the residual stream

As information moves through this model, it will be stored in 4 different subspaces.

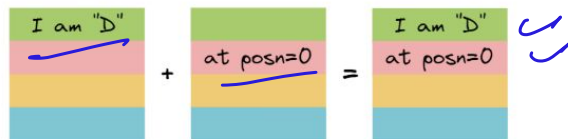
Each one has a different interpretation, so it will help to describe the subspaces in these terms:



To make the notation cleaner, I'll leave subspaces empty if they've not been used yet.

e.g. after adding token & positional encodings, we get

which I'll write as just



More notation: "A"^T is a one-hot encoded token vector, i.e. $\text{"A"}^T W_E = \begin{bmatrix} \text{I am "A"} \\ \text{ } \\ \text{ } \\ \text{ } \end{bmatrix}$

n^T is for a one-hot encoded position vector, i.e. $n^T W_{pos} = \begin{bmatrix} \text{ } \\ \text{at posn=n} \\ \text{ } \\ \text{ } \end{bmatrix}$

Subspaces in the Residual Stream

Information stored in the residual stream can be interpreted across four distinct subspaces:

Token Encoding Subspace

Interpretation:
“this token is x”

= rows of W_E

Positional Encoding Subspace

Interpretation:
“this token is at position x”

= rows of W_{pos}

Decoding Subspace

Interpretation:
“the next token will be x”

= cols of W_U

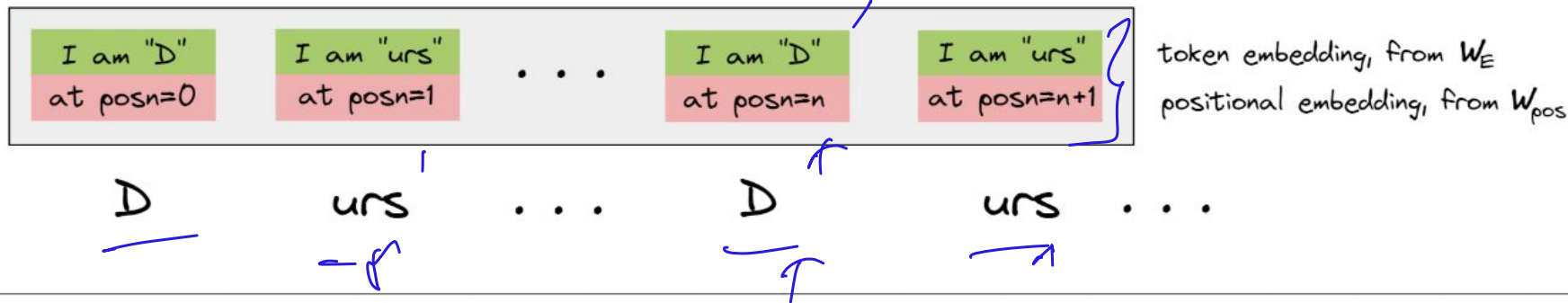
Prev Token Subspace

Interpretation:
“the previous token was x”

= “intermediate information”

An example: Embedding

Embedding



Subspaces in the Residual Stream

Information stored in the residual stream can be interpreted across four distinct subspaces:

Token Encoding Subspace

Interpretation:
"this token is x"

= rows of W_E

Positional Encoding Subspace

Interpretation:
"this token is at position x"

= rows of W_{pos}

Decoding Subspace

Interpretation:
"the next token will be x"

= cols of W_U

Prev Token Subspace

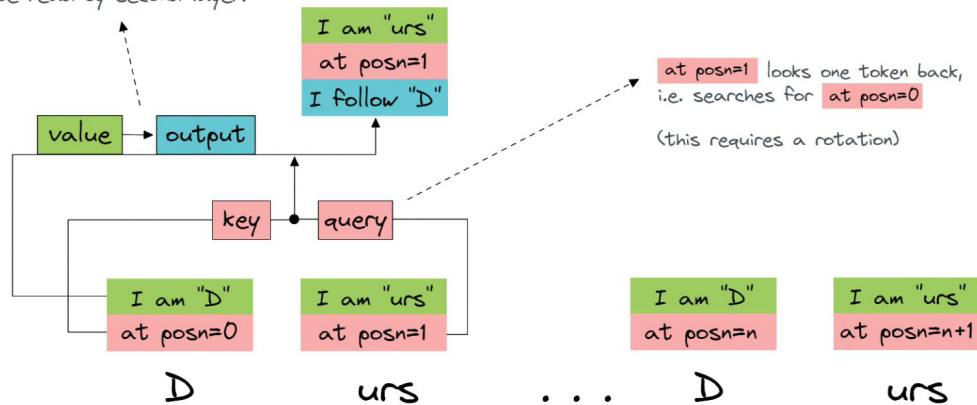
Interpretation:
"the previous token was x"

= "intermediate information"

Summary: each token looks one position backwards, and gets the information about which token preceded it.

```

graph LR
    value[value] -- solid arrow --> output[output]
    output -.-> stack
    subgraph stack
        direction TB
        A["I am "urs""]
        B["at posn=1"]
        C["I follow "D""]
    end
  
```



Query vector is $\begin{bmatrix} \text{I am "urs"} \\ \text{at posn}=1 \end{bmatrix}^T W_Q = \text{"I'm looking for posn } 1 - 1 = 0"$

Key vector is $\begin{bmatrix} \text{I am "D"} \\ \text{at posn=0} \end{bmatrix}^T W_K = \text{"I'm at posn 0"}$

i.e. we subtract one from the current position (this requires a rotation^[1])

The attention score (the dot product of the key and query vectors) is large, because the key is a good match for the query.

Query vector is $\begin{bmatrix} \text{I am "urs"} \\ \text{at posn}=1 \end{bmatrix}^T W_Q = \text{"I'm looking for posn } 1 - 1 = 0"$

Key vector is $\begin{bmatrix} \text{I am "D"} \\ \text{at posn}=0 \end{bmatrix}^T W_K = \boxed{\text{"I'm at posn 0"}}$

i.e. we subtract
one from the

The OV circuit reads tok and moves it to a differ (so it doesn't erase the embedding information which is stored at the destination token).

We have:

The OV circuit reads token information, and moves it to a different subspace (so it doesn't erase the token embedding information which is already stored at the destination token).

We have:

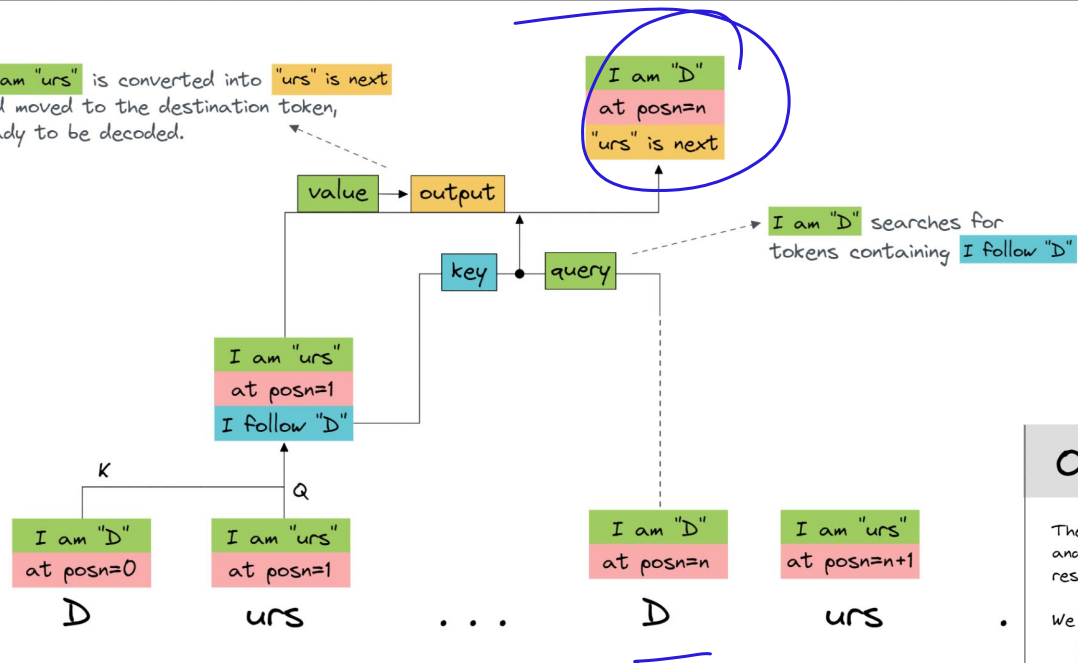
$W_V W_O =$

which gets added to the residual stream for the first "urs" token.

Layer 1 attention head

Summary: the 2nd "D" token looks back for the token following the 1st "D" (which is "urs"), and uses that as its prediction.

"I am 'urs'" is converted into "urs" is next and moved to the destination token, ready to be decoded.



QK circuit

The query is $\begin{bmatrix} \text{I am "D"} \\ \text{at posn=n} \end{bmatrix}^T W_Q = \text{"I'm looking for a token following "D"}$

The key is $\begin{bmatrix} \text{I am "urs"} \\ \text{at posn=1} \\ \text{I follow "D"} \end{bmatrix}^T W_K = \text{"I follow "D"}$

The attention score (the dot product of the key and query vectors) is large, because the key is a good match for the query.

OV circuit

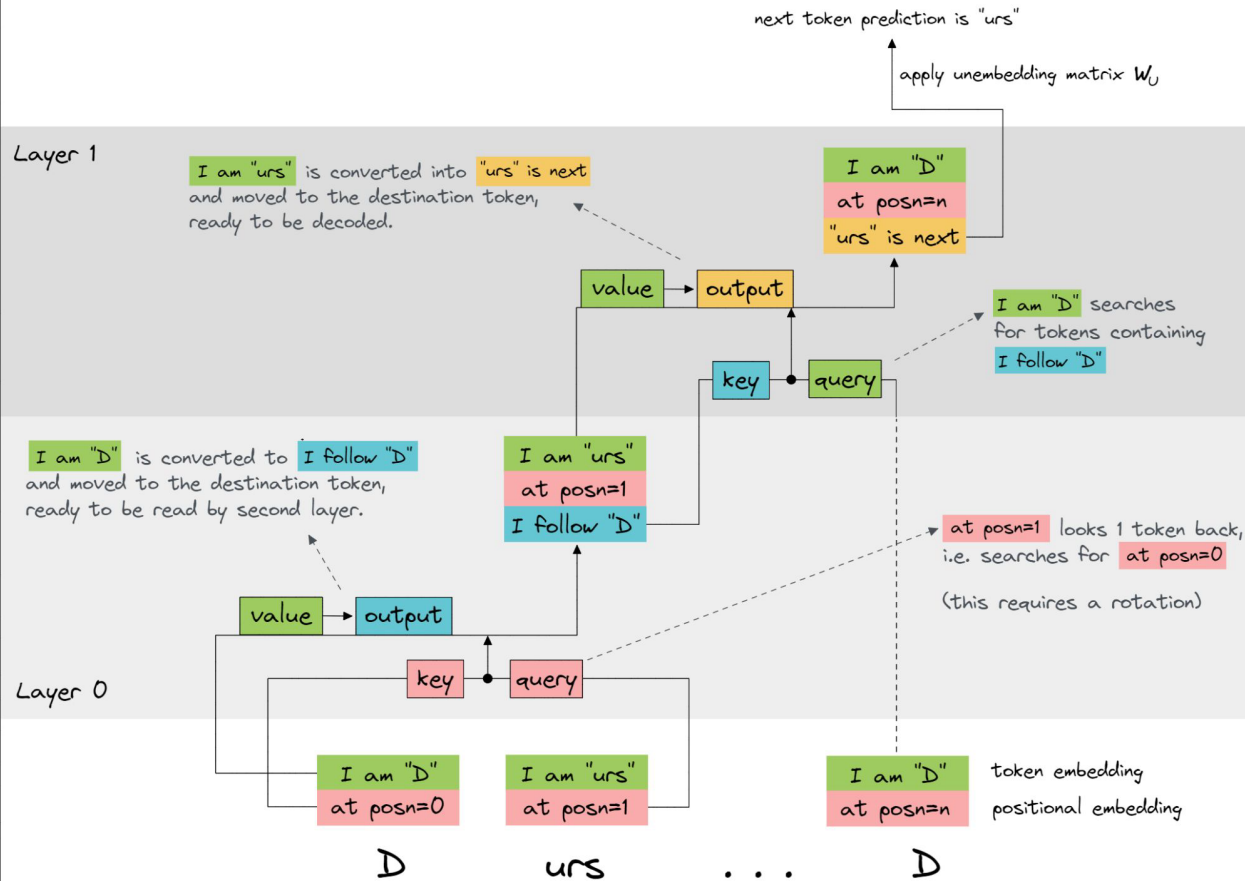
The OV circuit reads token information, and converts it into something which will result in a prediction of "urs".

We have:

$\begin{bmatrix} \text{I am "urs"} \\ \text{at posn=1} \\ \text{I follow "D"} \end{bmatrix}^T W_V W_O = \text{"urs" is next}$

which gets added to the residual stream of the second "D" token.

Putting everything together:



In-context Learning and Induction Heads

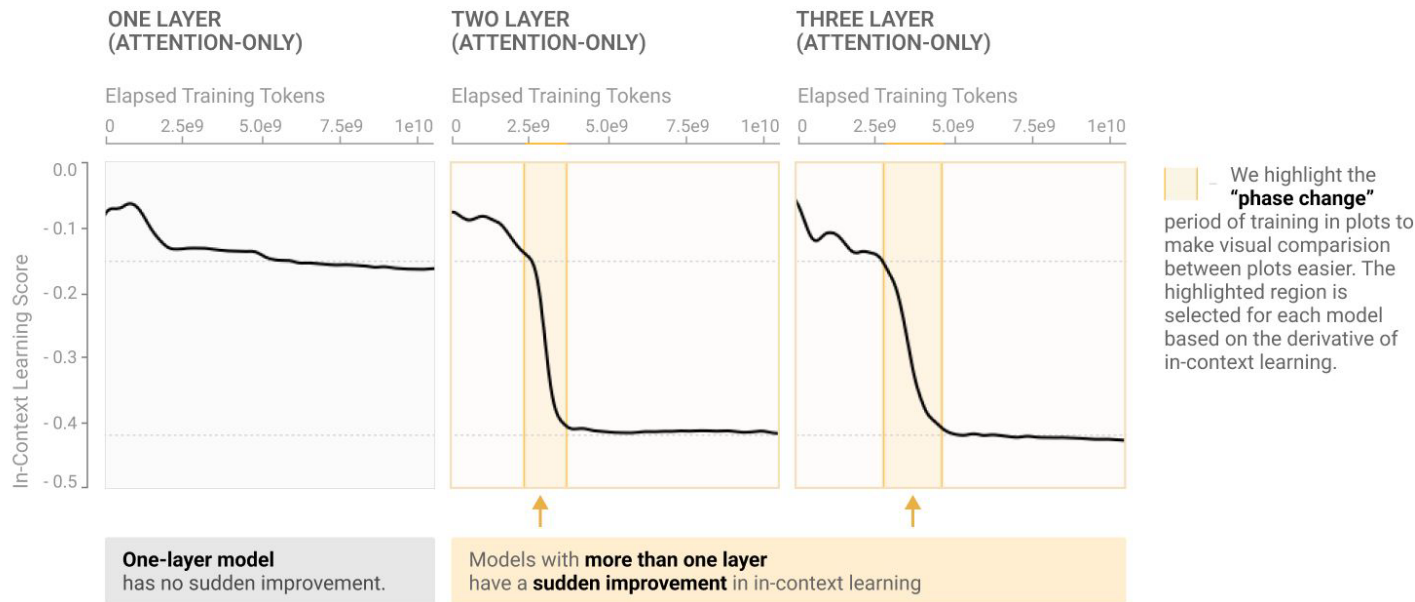
Argument 1 (*Macroscopic co-occurrence*): Transformer language models undergo a “phase change” early in training, during which induction heads form and simultaneously in-context learning improves dramatically.

Argument 2 (*Macroscopic co-perturbation*): When we change the transformer architecture in a way that shifts whether induction heads can form (and when), the dramatic improvement in in-context learning shifts in a precisely matching way.

Argument 3 (*Direct ablation*): When we directly “knock out” induction heads at test-time in small models, the amount of in-context learning greatly decreases.

Argument 4 (*Specific examples of induction head generality*): Although we define induction heads very narrowly in terms of copying literal sequences, we empirically observe that these same heads also appear to implement more sophisticated types of in-context learning, including highly abstract behaviors, making it plausible they explain a large fraction of in-context learning.

MODELS WITH MORE THAN ONE LAYER HAVE AN ABRUPT IMPROVEMENT IN IN-CONTEXT LEARNING

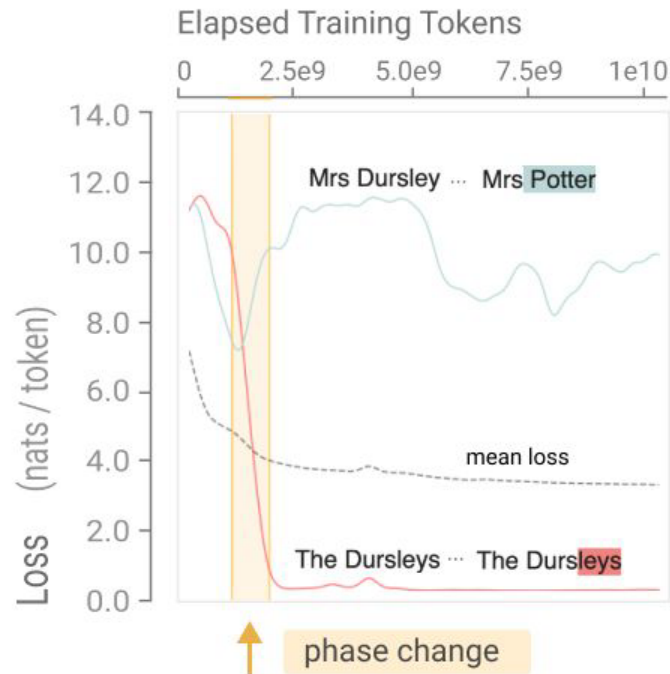


In-context learning score: the loss of the 500th token in the context minus the average loss of the 50th token in the context, averaged over dataset examples.

Why this choice? The 500th token is near the end of a length-512 context, and the 50th token is far enough into the context that some basic properties of the text have been established (such as language and document type) while still being near the start.

PER-TOKEN LOSSES OVER TRAINING

We look at two specific tokens in the Harry Potter passage a to highlight their movement during the phase change.



Mrs Potter : This token's loss increases during the phase change, because the model incorrectly predicts that the next token should be "D" from "Dursley".

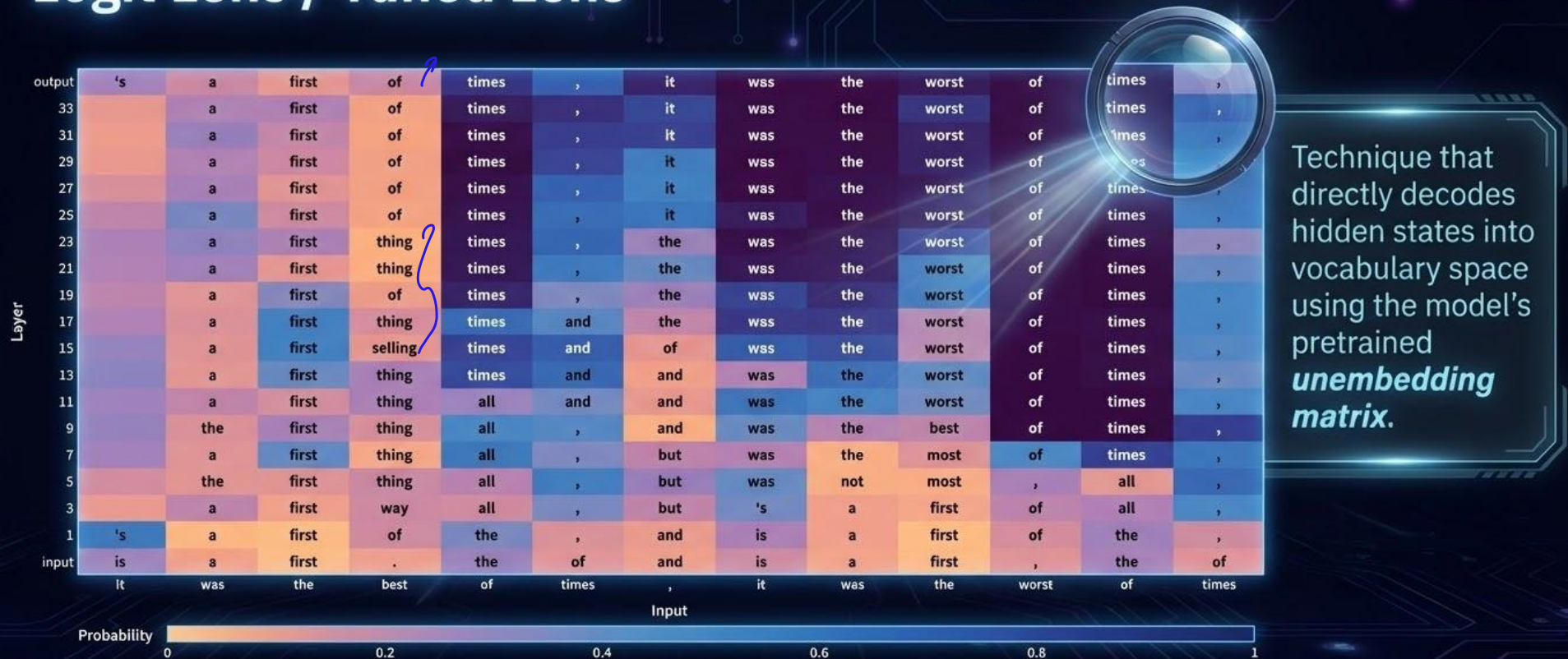
The Dursleys : This token's loss decreases during the phase change, because the model correctly predicts "leys" based on a previous example.

In-context Learning and Induction Heads

Argument 5 (*Mechanistic plausibility of induction head generality*): For small models, we can explain mechanistically how induction heads work, and can show they contribute to in-context learning. Furthermore, the actual mechanism of operation suggests natural ways in which it could be repurposed to perform more general in-context learning.

Argument 6 (*Continuity from small to large models*): In the previous arguments, the case for induction heads explaining in-context learning is stronger for small models than for large ones. However, many behaviors and data related to both induction heads and in-context learning are smoothly continuous from small to large models, suggesting the simplest explanation is that mechanisms are the same.

Logit Lens / Tuned Lens



Eliciting Latent Predictions from Transformers with the Tuned Lens. <https://arxiv.org/pdf/2303.08112>

Logit Lens: Basic Idea

GPT schematically looks like

- Project the input tokens from vocab space into the 1600-dim embedding space
- Modify this 1600-dim vector many times
- Project the final 1600-dim vector back into vocab space

We have a "dictionary," W , that lets us convert between vocab space and embedding space at any point. We know that *some* vectors in embedding space make sense when converted into vocab space:

- The very first embedding vectors are just the input tokens (in embedding space)
- The very last embedding vectors are just the output logits (in embedding space)

What about the 1600-dim vectors produced in the middle of the network, say the output of the 12th layer or the 33rd? If we convert them to vocab space, do the results make sense? The answer is ***yes***.

8/10/2021

Model's top-token and its logit



	(P) :	'we'	'train'	G	PT	:	?	(P) :	'an'	'aut'
h_out	''	'we'	'show'	'a'	'AN'	'models'	'based'	''	'a'	'N'
h46_out	''	'we'	'show'	'a'	'AN'	''	't'	''	'a'	'N'
h44_out	''	'we'	'show'	'a'	'BM'	'models'	'based'	'models'	'a'	'N'
h42_out	''	'we'	'show'	'a'	'rams'	'models'	'based'	'models'	'a'	'algorithms'
h40_out	''	'we'	'demonstrate'	'a'	'machine'	'models'	'based'	'models'	'a'	'algorithms'
h38_out	'we'	'we'	'demonstrate'	'neural'	'rap'	'models'	'based'	'models'	'a'	'algorithms'
h36_out	'we'	'we'	'demonstrate'	'neural'	'rap'	'models'	'based'	'models'	'a'	'algorithms'
h34_out	'we'	'we'	'demonstrate'	'models'	'rap'	'model'	'based'	'models'	'a'	'algorithms'
h32_out	'we'	'we'	'simulated'	'models'	'rap'	'model'	'based'	'models'	'a'	'adaptive'
h30_out	'targeted'	'we'	'found'	'a'	'rap'	''	'based'	''	'which'	'hybrid'
h28_out	'targeted'	'we'	'found'	'a'	'FP'	'Ms'	'based'	'rd'	'which'	'ambitious'
h26_out	'targeted'	'we'	'found'	'naïve'	'FP'	'Ms'	'based'	'rd'	'which'	'ambitious'
h24_out	'targeted'	'we'	'found'	'algorithms'	'FP'	's'	'based'	'rd'	'which'	'widely'
h22_out	'targeted'	'we'	'found'	'camp'	'FP'	's'	'based'	'rd'	'which'	'widely'
h20_out	'targeted'	'although'	'found'	'algorithm'	'FP'	'ouch'	'based'	'rd'	'000'	'single'
h18_out	'targeted'	'although'	'focus'	'camp'	'AP'	'ouch'	'based'	'rd'	'000'	'single'
h16_out	'targeted'	'unlike'	'focus'	'camp'	'MP'	'IME'	'based'	'rd'	'000'	'single'
h14_out	'targeted'	'note'	'target'	'camp'	'MS'	'IME'	'based'	'rd'	'000'	'single'
h12_out	'target'	'unlike'	'hope'	'split'	'MP'	'ouch'	'based'	'rd'	'000'	'massive'
h10_out	'updated'	'however'	'id'	'session'	'lott'	'IME'	'style'	'rd'	'000'	'massive'
h8_out	'target'	'however'	'target'	'evaluation'	'rom'	'IME'	'based'	'rd'	'000'	'enormous'
h6_out	'focused'	'however'	'id'	'ees'	'rou'	'pols'	'based'	'rd'	'which'	'enormous'
h4_out	'target'	'however'	'can'	'ees'	'rou'	'pols'	'sided'	'rd'	'and'	'enormous'
h2_out	'guid'	'and'	'Hardy'	'ees'	'rou'	'pols'	'based'	'rd'	'and'	'enormous'
h0_out	'chini'	'and'	'id'	'train'	'rou'	'pols'	'based'	'rd'	'and'	'isolated'

<https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>

The view till now focuses only on the top-1 guess at each layer, which is a reductive window on the full distributions.

Another way to look at things: we still reduce the *final* output to the top-1 guess, but we compare other distributions to the final one by looking at the rank of the final top-1 guess.

Even if the middle of the model hasn't yet converged to the final answer, maybe it's got that answer somewhere in its top 3, top 10, etc. That's a lot better than "top 50257."

(Remember: these are ranks of *the model's final top-1 prediction*, not *the true token*.)

model's top token and its rank over the ~50K vocab



<https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>

	'specificity'	'w'	'train'	'G'	'PT'	'2'	'3'	'4'	'ant'	'out'
h_out	1	1	1	1	1	1	1	1	1	1
h46_out	1	1	1	1	1	2	2	1	1	1
h44_out	1	1	1	1	2	1	1	2	1	1
h42_out	1	1	1	1	6	1	1	8	1	7
h40_out	1	1	2	1	44	1	1	4	1	10
h38_out	2	1	2	3	97	1	1	13	1	10
h36_out	2	1	3	4	137	1	1	45	1	16
h34_out	2	1	7	4	328	2	1	67	1	39
h32_out	2	1	13	3	580	2	1	48	1	87
h30_out	6	1	15	1	610	5	1	69	2	147
h28_out	6	1	16	1	642	4	1	66	3	125
h26_out	11	1	53	2	596	10	1	101	12	78
h24_out	18	1	36	6	917	14	1	80	13	75
h22_out	46	1	44	4	926	40	1	114	24	150
h20_out	132	3	145	5	847	64	1	208	61	368
h18_out	42	3	97	10	789	67	1	296	52	313
h16_out	85	3	140	11	1220	146	1	176	89	269
h14_out	115	6	424	20	1243	172	1	170	102	143
h12_out	187	12	1281	56	1985	322	1	391	89	167
h10_out	413	11	994	162	1819	493	2	213	139	164
h8_out	420	10	2338	213	1886	942	1	250	98	77
h6_out	994	7	4644	683	1660	1381	1	348	130	43
h4_out	1840	17	6128	1926	2542	2014	2	100	79	29
h2_out	1587	83	14270	1407	2000	1822	1	95	43	31
h0_out	20608	321	7507	4768	773	9399	1	486	105	59

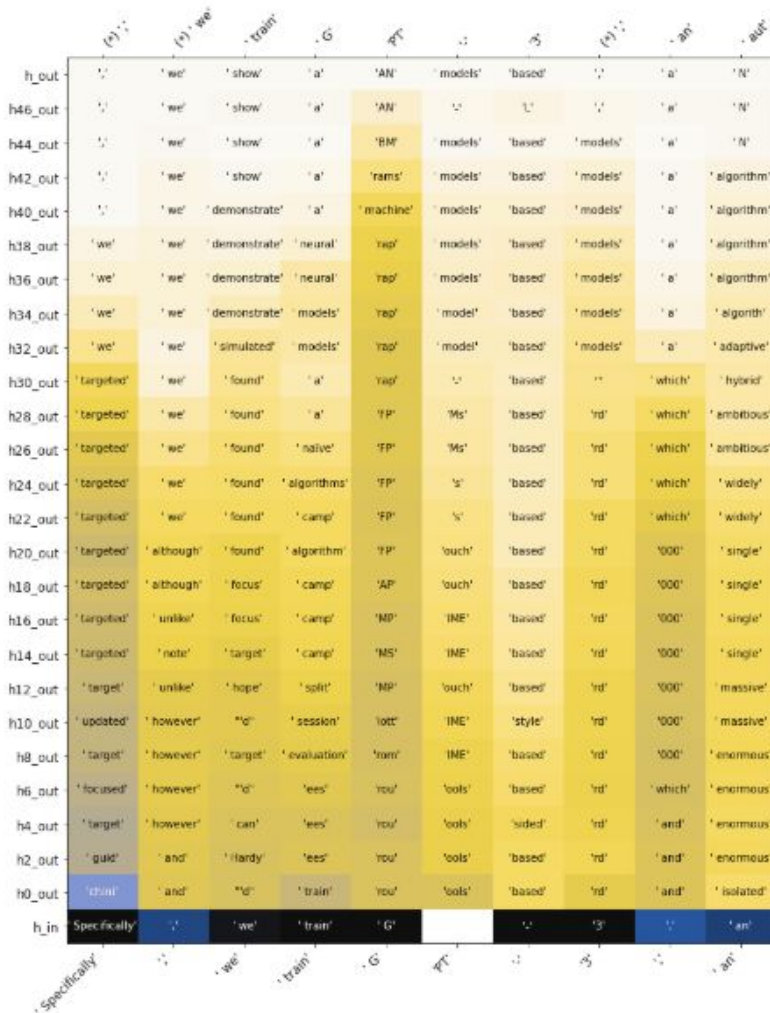
KL-divergence

Another way of comparing the similarity of two probability distributions is the KL divergence.

Taking the KL divergence of the intermediate probabilities w/r/t the final probabilities, we get a more continuous view of how the distributions smoothly converge to the model's output.

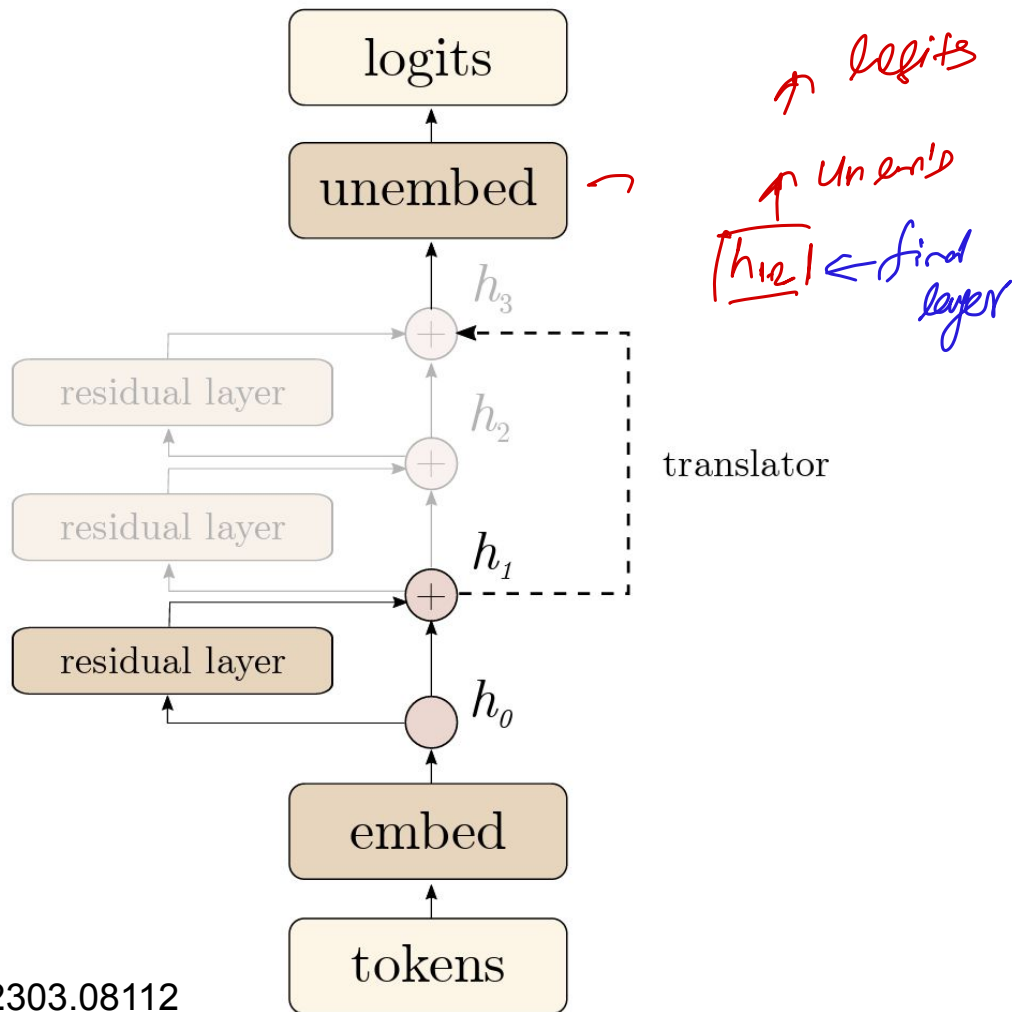
Because KL divergence is a more holistic measure of the similarity between two distributions than the ones

kl divergence from output distributions



<https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>

Tuned Lens



Logit Lens (theirs)

output	model	recurrent	architecture	that	called	attention	former	,
30	model	yet	that	that	called	attention	former	,
27	model	model	that	that	called	**	-	,
24	model	model	called	called	called	**	-	,
21	model	model	that	that	called	āG _i ,	āG _i ,	,
18	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,
15	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,
12	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,
9	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,
6	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,
3	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,	āG _i ,
input	new	simple	network	architecture	,	the	Trans	former

Tuned Lens

Figure 1. Comparison of our method, the *tuned lens* (bottom), with the “logit lens” (top) for GPT-Neo-2.7B prompted with an excerpt from the abstract of Vaswani et al. (2017). Each cell shows the top-1 token predicted by the model at the given layer and token index. The logit lens fails to elicit interpretable predictions before layer 21, but our method succeeds.

$$\text{TunedLens}_\ell(\mathbf{h}_\ell) = \text{LogitLens}(A_\ell \mathbf{h}_\ell + \mathbf{b}_\ell)$$

We refer to $(A_\ell, \mathbf{b}_\ell)$ as the *translator* for layer ℓ .

Loss function. We train the translators to minimize KL between the tuned lens logits and the final layer logits:

$$\argmin_{\mathbf{x}} \mathbb{E} [D_{KL}(f_{>\ell}(\mathbf{h}_\ell) || \text{TunedLens}_k(\mathbf{h}_\ell))]$$

output	model	recurrent	architecture	that	called	attention	former	,
30	model	model	architecture	that	called	attention	former	,
27	model	model	that	that	called	**	-	,
24	model	model	that	that	called	**	-	,
21	model	model	that	that	the	āG	-	,
18	model	model	that	that	the	so	-	,
15	method	method	that	that	which	so	-	,
12	method	-	that	that	which	so	-	,
9	and	and	that	for	which	result	-	,
6	a	to			and			
3	,	x	to	,	and	same	-	!
input	new	simple	network	architecture	,	the	Trans	former

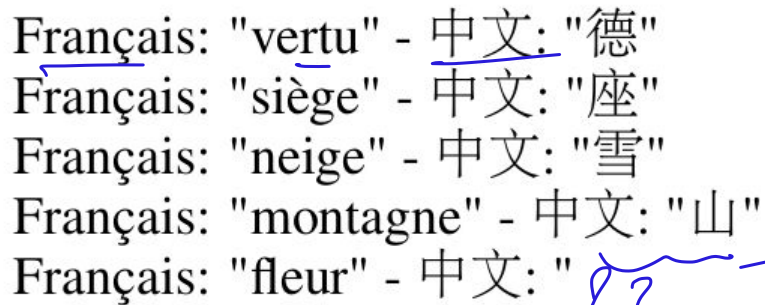
Input token

Probability



Translation Task

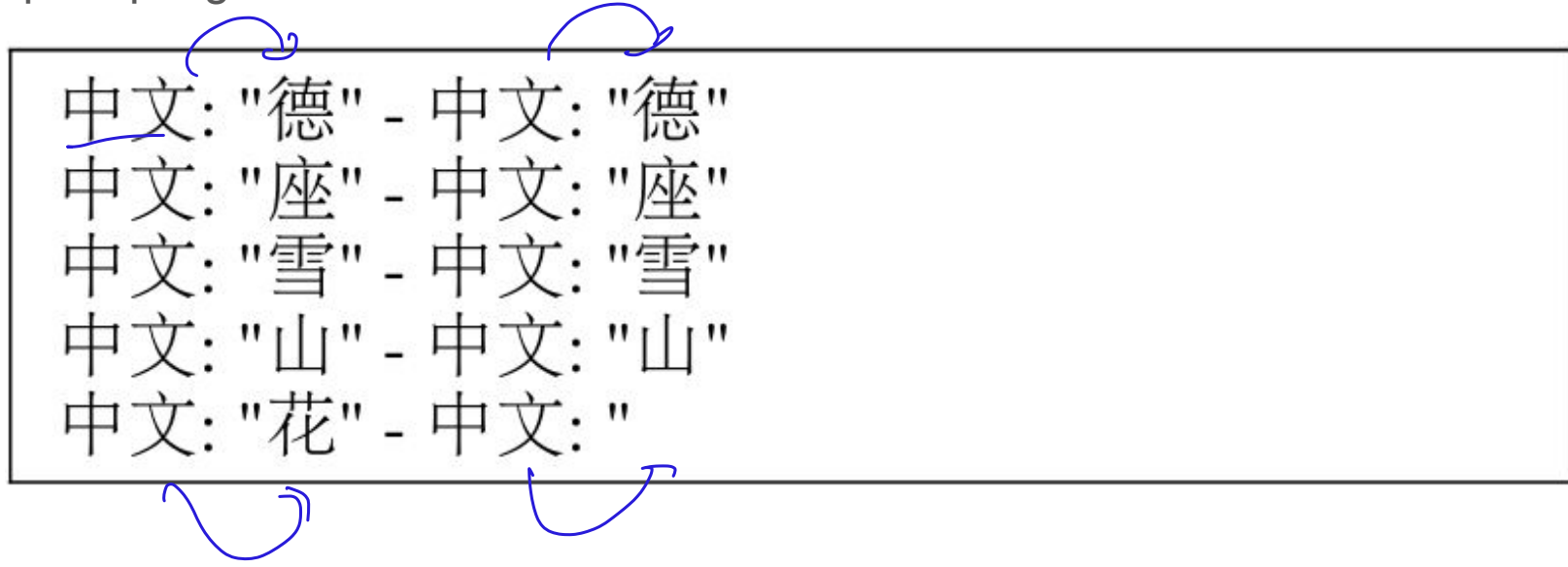
Here the task is to translate the preceding non-English (e.g., French) word to Chinese. We show the model four words with their correct translations, followed by a fifth word without its translation, and let the model predict the next token (“中文” means “Chinese” below):



Français: "vertu" - 中文: "德"
Français: "siège" - 中文: "座"
Français: "neige" - 中文: "雪"
Français: "montagne" - 中文: "山"
Français: "fleur" - 中文: "

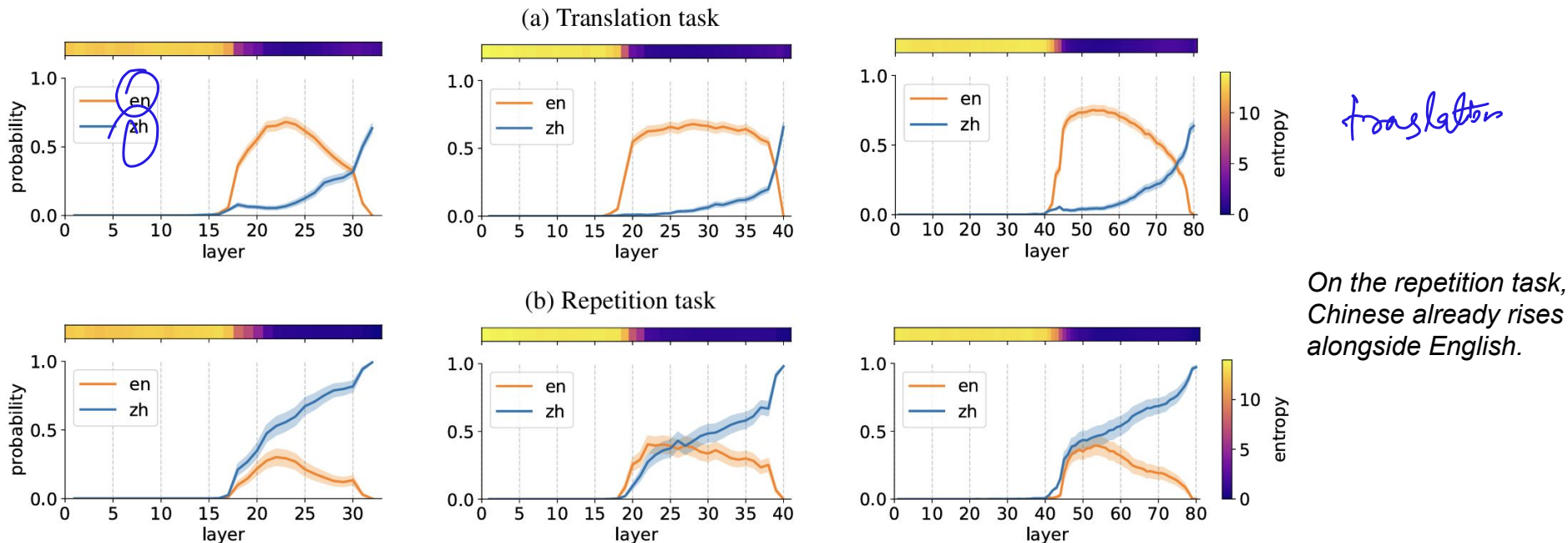
Repetition task

We task the model to simply repeat the last word, instead of translating it, by prompting as follows:



中文: "德" - 中文: "德"
中文: "座" - 中文: "座"
中文: "雪" - 中文: "雪"
中文: "山" - 中文: "山"
中文: "花" - 中文: "

Language probabilities for latents during Llama-2 forward



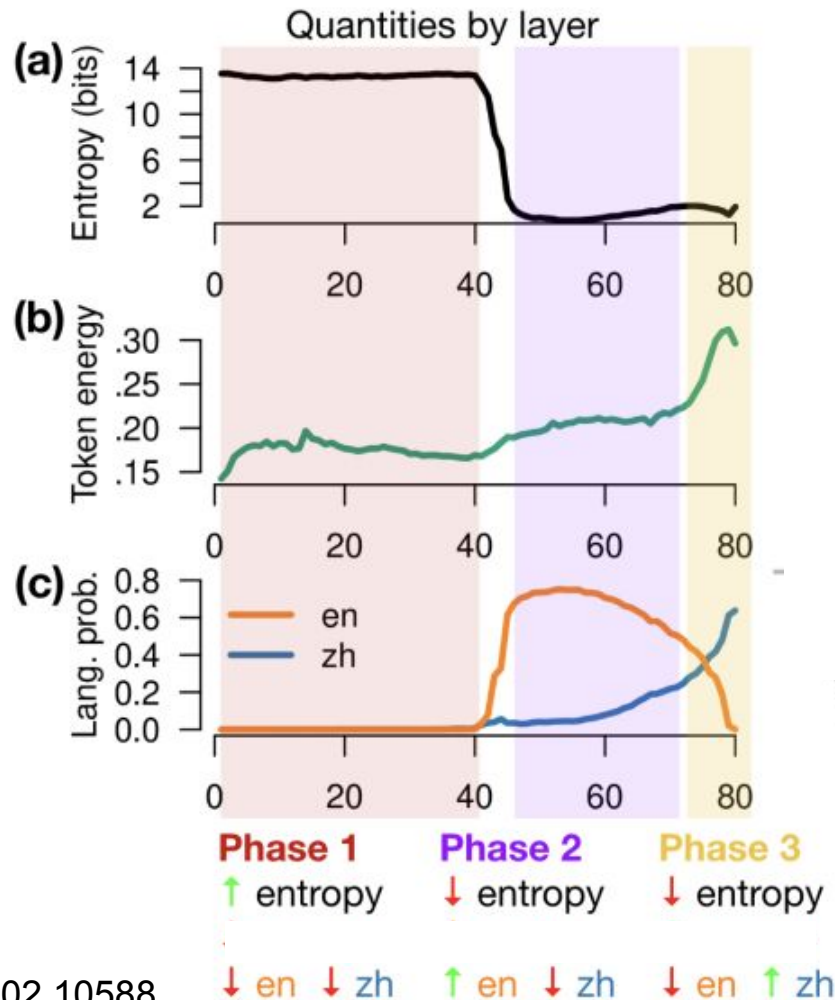
Neither the correct Chinese token nor its English analog garner any noticeable probability mass during the first half of layers. Then, around the middle layer, English begins a sharp rise followed by a decline, while Chinese slowly grows and, after a crossover with English, spikes on the last five layers.

Anatomy of forward pass

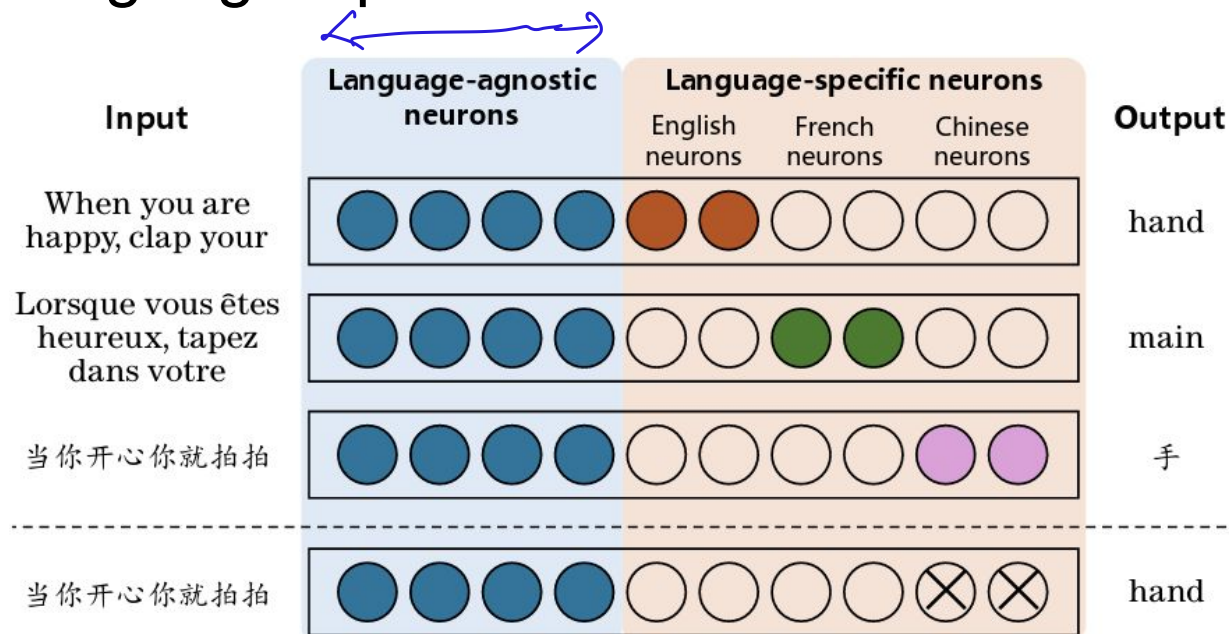
Phase I: Building good token representations

Phase II: Concept space with higher probability to tokens in English

Phase III: Predict concepts in the chosen target language



Language Specific Neurons



An illustration of region distribution of activated neurons when predicting the next word in language models across different languages. Here, colored circles denote activated neurons. When Chinese-specific neurons are deactivated (denoted by $\otimes$), the model may produce outputs in English.

LAPE Score (language activation probability entropy)

We feed existing LLMs with multilingual texts, each written in a single language.

For the j^{th} neuron in the i^{th} layer, we compute the activation probability when processing texts in language k :

$$p_{i,j}^k = \mathbb{E} \left(\mathbb{I}(\text{act_fn}(\tilde{\mathbf{h}}^i \mathbf{W}_1^i)_j > 0) \mid \text{language } k \right) \quad \mathbf{h}^i = \text{act_fn}(\tilde{\mathbf{h}}^i \mathbf{W}_1^i) \cdot \mathbf{W}_2^i$$

Handwritten notes: "self-att" and "RelU" with arrows pointing to the activation function and the weight matrix $\mathbf{W}_1^i$ respectively.

Distribution for each neuron, indicating its probability of activation for each language

$$\mathbf{p}_{i,j} = (p_{i,j}^1, \dots, p_{i,j}^k, \dots, p_{i,j}^l)$$

language activation
probability entropy

$$\text{LAPE}_{i,j} = - \sum_{k=1}^l p_{i,j}^k \log(p_{i,j}^k)$$

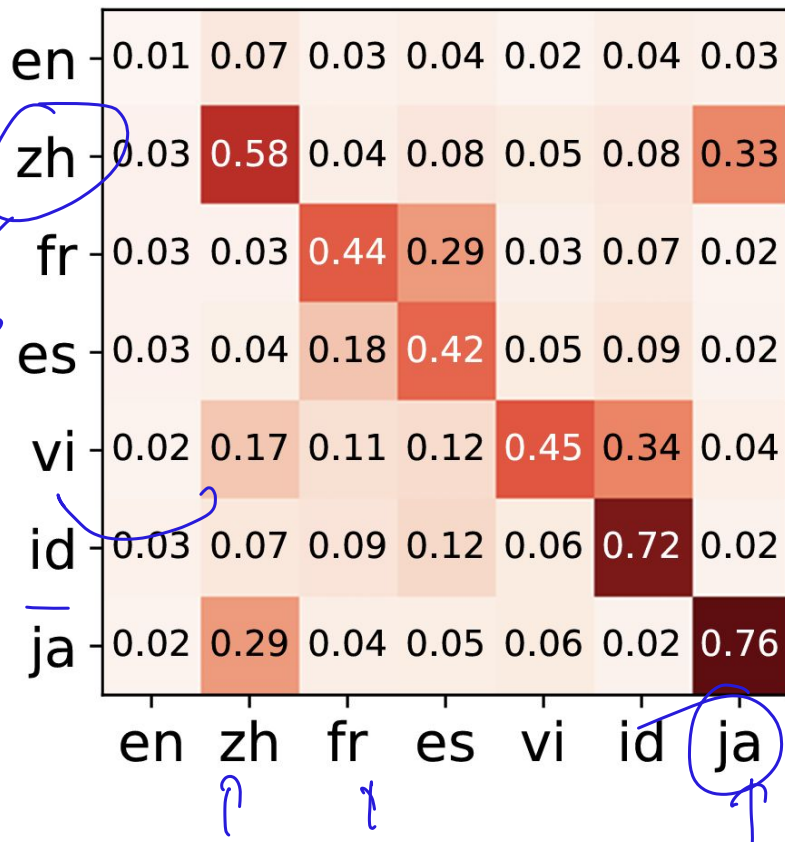
Handwritten blue squiggly line under the subscript i,j and a large blue curly bracket to the right of the equation.

We designate neurons with low LAPE scores as “language-specific neurons”

Deactivating language specific neurons

The element at the i -th row and j -th column is the perplexity increase for language j due to perturbations in a specific region of language i .

Roots
(deactivate zh neurons)
0-1%



Deactivating language specific neurons

Illustration of LLaMA-2-70B responses to a question in Simplified Chinese. The text in black is model's actual output and text in gray is our self-added translation.

The deactivated output is the generation when neurons of Simplified Chinese are deactivated.

Question

你是一位登上珠穆朗玛峰顶峰的登山者。描述一下你的情绪和从高处看到的景色。

(Translation: You are a mountain climber reaching the summit of Mount Everest. Describe your emotions and the view.)

Normal output

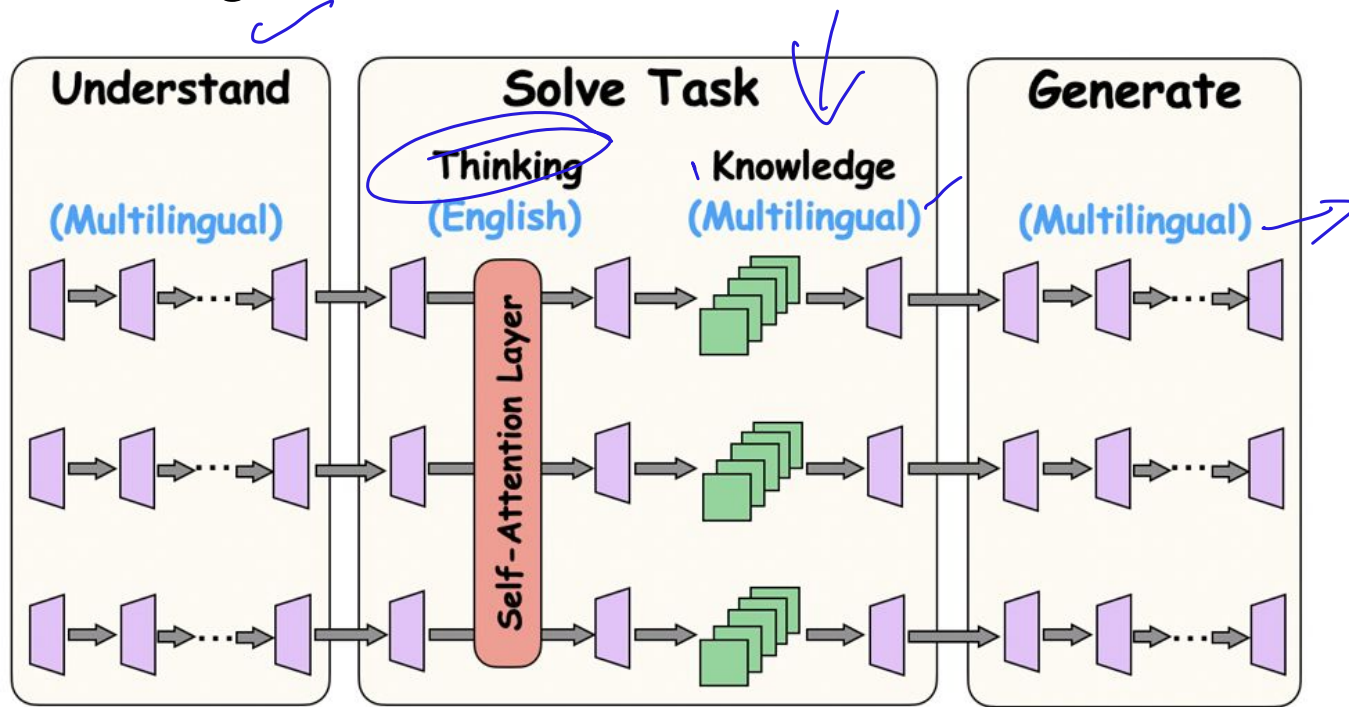
我是一个登上珠穆朗玛峰顶峰的登山者。当我站在山顶时，我感到非常兴奋和自豪。...

(Translation: I am a climber who has reached the summit of Mount Everest. When I stood on the top of the mountain, I felt very excited and proud. ...)

Deactivated output

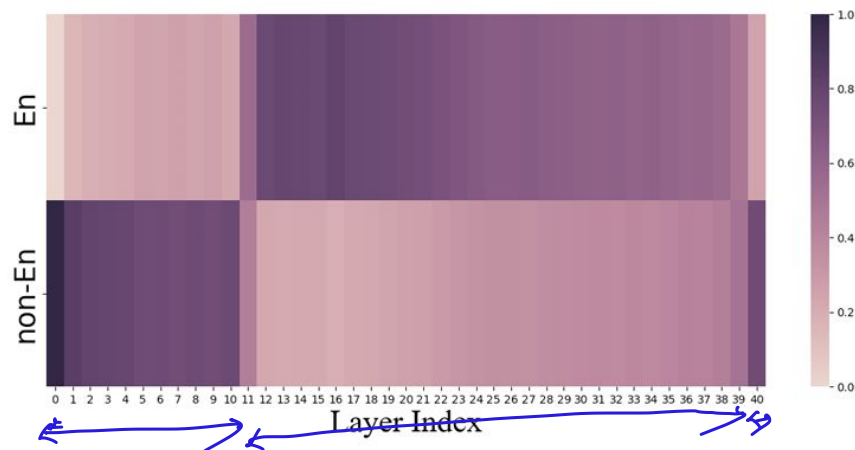
我是一個登上珠穆朗瑪峰頂峰的登山者。I am a mountaineer who has climbed to the top of Mount Everest. 當我站在珠my朗ma峰頂峰，我感到非常興奮和欣慰。...

Multilingual Workflow

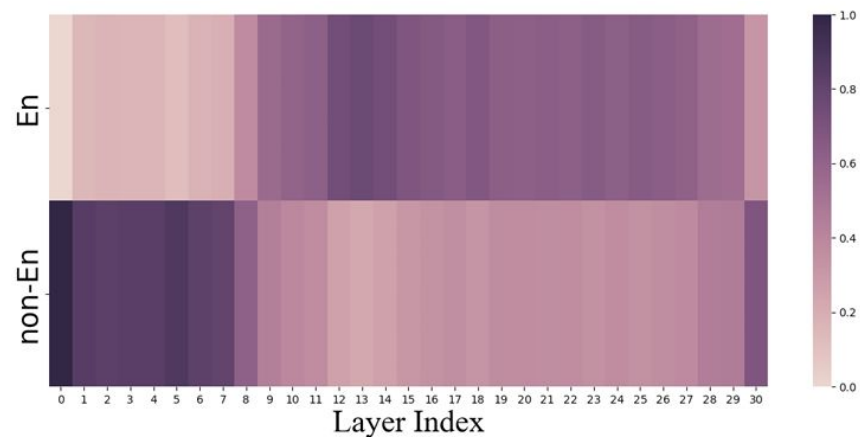


Understand the non-English query, interpret in English, solve the task in English and generate outputs in the original language

Processing non-English queries



(a) Vicuna-13b-v1.5



(b) BLOOMZ-7b1

We test LLMs with various non-English queries and decode the hidden embeddings of each layer to tokens within the LLM's vocabulary. Subsequently, we classify these decoded tokens into either English or non-English, and analyze the ratio.

Language-specific neuron (another method)

For a specific neuron within the i -th layer, denoted as $N(i)$ either located in the attention or feed-forward network, we quantify its importance in processing the input c by measuring the difference in the hidden embedding after the i -th layer, i.e., h_{i+1} , when $N(i)$ is activated or deactivated. Formally, the impact of neuron $N(i)$ for input c is defined as

$$\text{Imp}(N^{(i)} | c) = \|T_i \setminus N^{(i)}(h_i) - T_i(h_i)\|_2$$

$\nearrow$ layer parameters
 $h_i \xrightarrow{T_i} h_{i+1}$

we can obtain language-specific neurons that are important to all corpus in that language, i.e.,

$$\{N^{(i)} \mid \text{Imp}(N^{(i)} | c_l) \geq \epsilon, \forall c_l \in \mathcal{C}\}$$

$\downarrow$ language

$$\overline{h_i \xrightarrow{T_i \setminus N^{(i)}} h_{i+1}'} \\ \|h_{i+1}' - h_{i+1}\|_2$$

$$\geq \epsilon$$

$\rightarrow$ $N^{(i)}$ is important

Existence of language-specific neurons

We feed a corpus in a specific language to LLMs and identify neurons that are consistently activated, which are responsible for processing queries in that language.

We assess the performance of LLMs in corresponding languages when these neurons are deactivated versus when the same number of randomly sampled neurons are deactivated (0.13%).

Model	Method	Fr	Zh	Es	Ru	Avg.
Vicuna	Original	14.2	61.1	10.4	20.8	26.6
	Deactivate Random	14.1	61.6	10.4	20.8	26.7
	Deactivate Lang-Spec	0.83	0.00	0.24	0.42	0.37
Mistral	Original	15.2	56.4	10.6	21.0	25.8
	Deactivate Random	15.4	55.9	10.2	21.2	25.7
	Deactivate Lang-Spec	0.21	0.39	0.15	0.07	0.21

Multilingual workflow

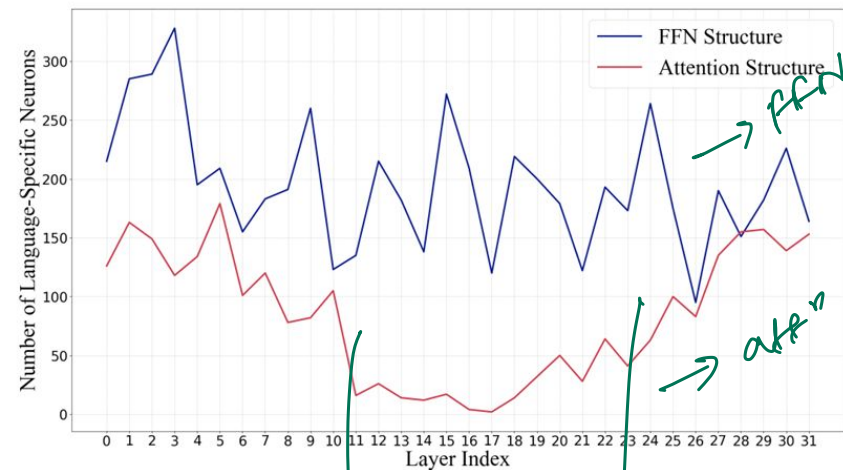


Figure 3: Number of language-specific neurons when processing multilingual queries.

LLMs first understand user input by unifying diverse linguistic features.

They then engage in the task-solving phase, employing English for reasoning (through self-attention) and leveraging multilingual knowledge (through feed-forward structures).
[Why? Will see in next topic]

Finally, the models generate responses aligned with the query's original language.